Universidade da Beira Interior
Departamento de Informática

2026: *Jogo de plataforma - Knights Journey*

Elaborado por:

Diogo Moreno, 43724

Orientador:

Professor Doutor Frutuoso G. M. Silva

14 de março de 2026

Agradecimentos

Gostaria de expressar o meu sincero agradecimento a todos aqueles que, de forma direta ou indireta, contribuíram para a realização deste projeto e para o meu percurso académico.

Em primeiro lugar, agradeço à minha família — em particular à minha mãe, ao meu pai e aos meus avós — pelo apoio constante, incentivo incondicional e compreensão ao longo de todo o meu percurso académico. O seu suporte foi fundamental para a concretização deste trabalho.

Aos meus colegas, deixo um agradecimento especial por proporcionarem um ambiente acolhedor, colaborativo e profissional, que me permitiu crescer tanto a nível académico como pessoal, tornando este percurso mais enriquecedor e motivador.

Agradeço ao Daniel Fernandes pela sua paciência, disponibilidade e apoio contínuo ao longo deste percurso, contribuindo para manter a motivação e o equilíbrio necessários durante os momentos mais exigentes.

Ao João Santos, agradeço pela companhia e partilha ao longo da vida académica, incluindo os momentos de convívio que ajudaram a aliviar a pressão e a fortalecer laços importantes durante esta etapa.

Por fim, agradeço ao Bruno Marinho, colega de universidade, pelo companheirismo tanto no contexto académico como no desporto universitário, experiências que contribuíram de forma significativa para o meu desenvolvimento pessoal e académico.

A todos, o meu profundo e sincero obrigado.

Conteúdo

Conteúdo iii

Lista de Figuras v

1 Introdução 1
1.1 Enquadramento 1
1.2 Motivação UBI 2
1.3 Objetivos 2
1.4 Organização do Documento 3

2 Estado da Arte 5
2.1 Introdução 5
2.2 Evolução dos Jogos de Plataforma 2D 5
2.3 Princípios de *Game Design* em Plataformas 6
2.4 *Pixel Art* e Estética Retro 6
2.5 Ferramentas de Desenvolvimento *Indie* 6
2.6 Conclusões 7

3 Tecnologias e Ferramentas Utilizadas 9
3.1 Introdução 9
3.2 *Godot Engine* 9
3.3 Recursos Visuais 10
3.3.1 *Brackeys Platformer Bundle* 10
3.3.2 *Backgrounds* com *Parallax Effect* 11
3.4 Recursos Auditivos 11
3.5 Justificação das Escolhas Tecnológicas 12
3.6 Conclusões 12

4 Implementação e Testes 13
4.1 Introdução 13
4.2 Movimento do Jogador e *Ice Level* 13
4.2.1 Movimento base e resposta ao *input* 14
4.2.2 Introdução do *Ice Level* 14

4.2.3 Movimento em superfícies de gelo 15
4.2.4 Comparação com movimento normal 16
4.2.5 Impacto no *game feel* e testes 16
4.3 Sistema de Som e Áudio 17
4.3.1 Música Ambiente por Nível 17
4.3.2 Efeitos Sonoros e *Pickups* 18
4.3.3 Gestão de Áudio com *Audio Buses* 19
4.4 *Parallax Effect* 20
4.4.1 *Scripts* de Movimento das Camadas 21
4.5 *Pickups* e Sistema de Recolha 22
4.6 *KillZones* e *Restart* do Nível 24
4.7 Armadilhas Dinâmicas: *Stalactites* 25
4.7.1 Estado inicial e preparação 26
4.7.2 Deteção do jogador e aviso prévio 26
4.7.3 Queda e libertação da física 27
4.7.4 Impacto no design de níveis 28
4.8 Gestão e Transição de Níveis 28
4.8.1 Ciclo de vida dos níveis e nó `LevelRoot`. 29
4.8.2 Transições com efeito de *fade*. 29
4.8.3 Cena de introdução e início do jogo. 30
4.8.4 Configuração do nível e ligação de eventos. 30
4.8.5 Progressão entre níveis e créditos finais. 31
4.9 Testes e Validação 31
4.9.1 Versão Inicial do Protótipo 31
4.9.2 Segunda Versão: Estruturação do Jogo 32
4.9.3 Versão Final: Consolidação e Refinamento 33
4.9.4 Considerações Finais sobre os Testes 33
4.10 Conclusões 34

5 Conclusões e Trabalho Futuro 35
5.1 Conclusões Principais 35
5.2 Trabalho Futuro 36
5.2.1 Sistema de Menus e Interface 37
5.2.2 Expansão de Inimigos e Desafios 37
5.2.3 Novas Mecânicas de Movimento 37
5.2.4 Aplicações e Extensões do Projeto 37
5.3 Considerações Finais 38

Bibliografia 39

Lista de Figuras

4.1 Comparação entre um nível com superfícies normais e um nível de gelo (*Ice Level*). 14
4.2 Conjunto de faixas de música ambiente utilizadas nos diferentes níveis. 17
4.3 Nodo de música ambiente integrado na estrutura do nível. 18
4.4 Ficheiros de *sound effects* (SFX) utilizados para ações do jogador e *pickups*. 18
4.5 Configuração dos *Audio Buses* para música e SFX. 19
4.6 Exemplo das camadas de fundo utilizadas no efeito de *parallax*. . . 20
4.7 Estrutura do efeito de *parallax* na árvore de cenas do *Godot Engine*. 20
4.8 Exemplo de moedas posicionadas ao longo de um nível. 22
4.9 Exemplo de posicionamento de uma *KillZone* na zona inferior do nível. 24
4.10 Posicionamento de uma stalactite no teto do nível com área de deteção do jogador. 26
4.11 Organização dos níveis como cenas independentes. 28
4.12 Protótipo inicial do jogo com um único nível e mecânicas base de plataformas 2D. 31
4.13 Evolução do jogo com introdução de *killzones* mais elaboradas e estruturação dos níveis. 32
4.14 Cena de introdução e cena de créditos da versão final do jogo. . . . 33

Lista de Excertos de Código

4.1 Alternância do estado de movimento em superfícies de gelo. . . 15
4.2 Movimento horizontal do jogador em superfícies de gelo. 15
4.3 Movimento horizontal do jogador em superfícies normais. . . . 16
4.4 Reprodução do SFX aquando da recolha de uma moeda. 19
4.5 Script `cloudLayer1st.gd` responsável pelo movimento das camadas mais distantes. 21
4.6 Script `cloudLayer.gd` responsável pelo movimento das camadas mais próximas. 21
4.7 Implementação da moeda e emissão do sinal de recolha. 23
4.8 Implementação da *KillZone* com efeito de *slow-motion* e *restart* do nível. 25
4.9 Configuração inicial da *stalactite*. 26
4.10 Ativação da *stalactite* ao detetar o jogador. 27
4.11 Efeito visual de aviso antes da queda. 27
4.12 Libertação da *stalactite* e início da queda. 27
4.13 Carregamento dinâmico e substituição do nível atual. 29
4.14 Implementação do efeito de *fade* entre níveis. 29
4.15 Ligação do sinal de início da cena de introdução. 30
4.16 Configuração do nível e ligação de sinais. 30

Acrónimos

CFIUTE Centro de Formação Interação UBI Tecido Empresarial

UBI Universidade da Beira Interior

MIT Massachusetts Institute of Technology

SFX *sound effects*

Capítulo

1

Introdução

1.1 Enquadramento

Este relatório foi desenvolvido no âmbito da unidade curricular de Projeto da Universidade da Beira Interior (Universidade da Beira Interior (UBI)), sendo todo o trabalho realizado nas infraestruturas da instituição, em articulação com o Centro de Formação Interação UBI Tecido Empresarial (Centro de Formação Interação UBI Tecido Empresarial (CFIUTE)).

Os jogos de plataforma 2D constituem um dos géneros mais icónicos e duradouros da indústria de videojogos. Desde os primeiros títulos de referência, como *Super Mario Bros.* e *Sonic the Hedgehog*, o género tem evoluído de forma consistente, mantendo a sua relevância através da combinação de mecânicas simples de aprender com desafios progressivamente mais exigentes [1, 2]. Esta dualidade entre acessibilidade e profundidade torna os jogos de plataforma particularmente apelativos tanto para jogadores casuais como para jogadores experientes.

Do ponto de vista académico e formativo, os jogos de plataforma 2D revelam-se especialmente adequados para projetos de desenvolvimento de jogos. A sua simplicidade conceptual permite focar a atenção em conceitos fundamentais como física do movimento, deteção e resposta a colisões, *design* de níveis e experiência do utilizador, sem a complexidade adicional associada a ambientes tridimensionais [3]. Adicionalmente, estudos indicam que o desenvolvimento de jogos promove competências técnicas e criativas, bem como pensamento crítico e resolução de problemas, sendo uma ferramenta eficaz em contextos educacionais [4].

Neste contexto, o projeto *Knights Journey* surgiu da vontade de criar uma experiência de jogo envolvente que combinasse elementos clássicos dos jo-

gos de plataforma 2D com uma ambientação medieval. O projeto explora as capacidades do motor de jogo *Godot Engine* no desenvolvimento 2D, permitindo aplicar princípios teóricos de *game design* e programação na construção de um produto interativo funcional. A escolha deste motor deve-se à sua natureza *open-source*, à documentação extensa e à sua adequação a projetos académicos e de desenvolvimento independente [5].

1.2 Motivação UBI

A motivação principal para o desenvolvimento de *Knights Journey* surge do interesse pessoal em *game design* e da necessidade de compreender os processos técnicos e criativos envolvidos na criação de um jogo completo. O género de plataforma 2D oferece um equilíbrio ideal entre complexidade técnica e viabilidade de implementação, permitindo focar em aspetos fundamentais como movimento responsivo, *design* de níveis coeso e *feedback* visual e auditivo.

Adicionalmente, a escolha do *Godot Engine* como plataforma de desenvolvimento foi motivada pela sua natureza *open-source*, documentação extensa e comunidade ativa, tornando-o uma ferramenta acessível e poderosa para desenvolvimento *indie*. A temática medieval do jogo permitiu explorar uma estética visual distintiva através de *pixel art*, um estilo artístico que combina nostalgia com expressividade criativa.

1.3 Objetivos

Os objetivos principais deste projeto são:

1. Desenvolver um jogo de plataforma 2D funcional e jogável com múltiplos níveis progressivamente desafiantes.
2. Implementar mecânicas de movimento fluido e responsivo que proporcionem uma experiência de jogo satisfatória.
3. Integrar elementos visuais e auditivos coesos que reforcem a ambientação medieval e a imersão do jogador.
4. Aplicar princípios de *game design* na construção de níveis que equilibrem acessibilidade e desafio.
5. Dominar as funcionalidades do *Godot Engine* relevantes para desenvolvimento de jogos 2D, incluindo sistema de cenas, física 2D e gestão de recursos.

6. Criar uma experiência de jogo completa que demonstre competências técnicas e criativas em desenvolvimento de videojogos.

1.4 Organização do Documento

De modo a refletir o trabalho que foi feito, este documento encontra-se estruturado da seguinte forma:

1. O primeiro capítulo – **Introdução** – apresenta o projeto *Knights Journey*, a motivação para a sua escolha, o enquadramento do mesmo no contexto de jogos de plataforma 2D, os seus objetivos e a respetiva organização do documento.

2. O segundo capítulo – **Estado da Arte** – explora a evolução histórica dos jogos de plataforma 2D, analisa títulos de referência no género, discute princípios fundamentais de *game design* aplicáveis a plataformas e examina tendências contemporâneas no desenvolvimento *indie*.

3. O terceiro capítulo – **Tecnologias e Ferramentas Utilizadas** – descreve detalhadamente o *Godot Engine* e as suas capacidades para desenvolvimento 2D, apresenta os recursos visuais e auditivos utilizados no projeto e justifica as escolhas tecnológicas realizadas.

4. O quarto capítulo – **Implementação e Testes** – documenta o processo de desenvolvimento do jogo, desde a conceção inicial até à implementação final, descrevendo as mecânicas implementadas, o *design* dos níveis, os testes realizados e os desafios encontrados.

5. O quinto capítulo – **Conclusões e Trabalho Futuro** – sintetiza as principais conclusões do projeto, reflete sobre os objetivos alcançados e identifica possibilidades de expansão e melhoramento futuro.

Capítulo

2

Estado da Arte

2.1 Introdução

Este capítulo explora o contexto teórico e prático no qual se insere o desenvolvimento de *Knights Journey*. Começamos por analisar a evolução histórica dos jogos de plataforma 2D, identificando marcos importantes e padrões de *design* que definiram o género. Seguidamente, examinamos princípios fundamentais de *game design* específicos a jogos de plataforma, incluindo conceitos como *level design*, *player feedback* e curva de dificuldade. Por fim, discutimos o panorama atual do desenvolvimento *indie* e as ferramentas que tornaram a criação de jogos mais acessível.

2.2 Evolução dos Jogos de Plataforma 2D

Os jogos de plataforma 2D representam um dos géneros fundacionais da indústria de videojogos. O género estabeleceu-se na década de 1980 com títulos como ***Donkey Kong*** (1981) e ***Super Mario Bros*** (1985), que definiram mecânicas essenciais como salto preciso, navegação vertical e horizontal, e progressão através de níveis estruturados [6, 7].

Durante a década de 1990, o género atingiu o seu apogeu com obras como ***Sonic the Hedgehog***, que introduziu velocidade e *momentum* como elementos centrais de *gameplay*, e ***Super Metroid***, que popularizou a exploração não-linear e a aquisição gradual de habilidades [1, 8].

O advento dos gráficos 3D nos anos 2000 levou a um declínio temporário do género, mas a revolução *indie* da década de 2010 trouxe um renascimento significativo. Títulos como ***Celeste***, ***Hollow Knight*** e ***Dead Cells*** demonstraram que plataformas 2D continuam capazes de inovar, combinando mecâni-

cas clássicas com narrativas mais maduras, sistemas de progressão complexos e desafios exigentes [2, 9].

2.3 Princípios de *Game Design* em Plataformas

O sucesso de um jogo de plataforma depende fundamentalmente da qualidade do seu *game feel* — a sensação de controlar a personagem. Este conceito engloba tempo de resposta aos *inputs*, física de movimento (aceleração, desaceleração e gravidade) e *feedback* visual e auditivo que confirma as ações do jogador [1, 2].

O *level design* em plataformas segue princípios específicos. Os níveis funcionam como tutoriais implícitos, introduzindo mecânicas gradualmente através de situações de baixo risco antes de as combinar em desafios mais complexos [10].

A curva de dificuldade deve ser cuidadosamente balanceada. Desafios iniciais estabelecem competência básica, enquanto níveis posteriores requerem domínio e combinação de habilidades. Este equilíbrio é essencial para manter o jogador num estado de *flow*, conforme definido por Csikszentmihalyi [11].

2.4 *Pixel Art* e Estética Retro

A *pixel art* emergiu como estilo artístico por necessidade técnica nas primeiras gerações de consolas, mas transformou-se numa escolha estética deliberada no desenvolvimento contemporâneo [12].

A estética retro evoca nostalgia enquanto permite experimentação moderna. Técnicas como *parallax scrolling* e *particle effects* adicionam complexidade visual mantendo a identidade *pixel art* [13]. A escolha da paleta de cores afeta significativamente o *mood* e a atmosfera, com tons quentes criando ambientes acolhedores e tons frios sugerindo perigo ou mistério.

2.5 Ferramentas de Desenvolvimento *Indie*

O desenvolvimento de jogos democratizou-se significativamente na última década através de motores de jogo acessíveis. *Unity* e *Unreal Engine* dominam o mercado comercial [14, 15], mas alternativas como o *Godot Engine* ganharam relevância por serem totalmente gratuitas e *open-source* [5, 3].

Plataformas de distribuição de *assets* como itch.io[16] revolucionaram o acesso a recursos artísticos e sonoros. Artistas disponibilizam *sprite packs*, *ti-*

lesets, músicas e efeitos sonoros sob licenças permissivas, permitindo que *developers* independentes acedam a conteúdo de qualidade profissional. Esta democratização de recursos é fundamental para o ecossistema *indie*, permitindo que projetos pequenos alcancem *polish* visual e auditivo anteriormente restrito a grandes estúdios.

2.6 Conclusões

O panorama atual dos jogos de plataforma 2D é caracterizado por diversidade e inovação. O género mantém-se vital através da combinação de mecânicas estabelecidas com novas ideias, acessibilidade de ferramentas de desenvolvimento e uma comunidade ativa de *developers* e jogadores. *Knights Journey* insere-se neste contexto procurando honrar tradições do género enquanto explora possibilidades criativas através do *Godot Engine* e recursos da comunidade *indie*.

Capítulo

3

Tecnologias e Ferramentas Utilizadas

3.1 Introdução

Este capítulo detalha as tecnologias e ferramentas utilizadas no desenvolvimento de *Knights Journey*. Começamos por apresentar o *Godot Engine*, o motor de jogo escolhido para o projeto, explorando as suas características e capacidades específicas para desenvolvimento 2D. Seguidamente, descrevemos os recursos visuais e auditivos integrados no jogo, incluindo *sprite packs*, *backgrounds* e música. Por fim, justificamos as escolhas tecnológicas realizadas e como estas contribuíram para o desenvolvimento eficiente do projeto.

3.2 *Godot Engine*

O *Godot Engine* [5] é um motor de jogo *open-source* lançado em 2014, que se tornou uma alternativa popular a motores comerciais como *Unity* e *Unreal Engine*. A escolha do *Godot* para *Knights Journey* baseou-se em vários fatores fundamentais.

Primeiramente, sendo completamente gratuito e *open-source* sob licença de Massachusetts Institute of Technology (MIT), o *Godot* elimina preocupações com *licensing* e custos, tornando-o ideal para projetos académicos e desenvolvimento *indie*. A comunidade ativa contribui constantemente com melhorias, documentação e tutoriais, facilitando a aprendizagem.

A arquitetura de cenas do *Godot Engine* é particularmente elegante para desenvolvimento 2D. Cada elemento do jogo é uma cena (*scenes*) que pode conter outras cenas, permitindo estruturação hierárquica e reutilização mo-

dular. Uma personagem pode ser uma cena independente que é instanciada em diferentes níveis, facilitando manutenção e iteração.

Um dos conceitos mais relevantes do *Godot Engine* é o *scene nesting*, isto é, a possibilidade de instanciar uma cena dentro de outra cena. Este mecanismo permite que cenas funcionem como blocos reutilizáveis, promovendo modularidade, organização e separação de responsabilidades. Por exemplo, um personagem, uma armadilha ou um elemento de interface pode ser desenvolvido como uma cena independente e posteriormente integrado em diferentes níveis ou contextos do jogo.

O *scene nesting* facilita a reutilização de lógica e recursos, reduzindo duplicação de código e simplificando a manutenção do projeto. Alterações realizadas numa cena base propagam-se automaticamente a todas as instâncias onde essa cena é utilizada, o que é particularmente vantajoso em projetos com múltiplos níveis e elementos repetidos, como é o caso de *Knights Journey*.

O motor oferece um sistema de física 2D robusto com suporte nativo para colisões, *rigid bodies*, *kinematic bodies* e áreas. Para jogos de plataforma, o nodo *KinematicBody2D* é especialmente relevante, oferecendo controlo preciso sobre movimento e colisões com funções como *move_and_slide()*, que simplificam a implementação de mecânicas de movimento.

A linguagem *GDScript*, similar a *Python*, apresenta uma curva de aprendizagem suave com sintaxe clara e integração profunda com o motor. Alternativamente, o *Godot* suporta C# e C++, oferecendo flexibilidade conforme as necessidades de *performance*.

3.3 Recursos Visuais

3.3.1 *Brackeys Platformer Bundle*

Os recursos visuais foram obtidos a partir do *Brackeys Platformer Bundle* [17], disponível em itch.io [16], e serviram como base visual fundamental para *Knights Journey*. Este *pack* contém *sprites* de personagens, *tiles* para construção de níveis, objetos interativos e elementos de decoração, todos coerentes esteticamente em estilo *pixel art*.

A utilização deste *bundle* acelerou significativamente o desenvolvimento ao fornecer *assets* prontos para uso, permitindo focar em mecânicas de jogo e *level design*. Os *sprites* incluem animações completas para estados como *idle*, *walking* e *jumping*, essenciais para *feedback* visual responsivo.

Os *tilesets* modulares permitem construção flexível de níveis. *Tiles* individuais podem ser combinados de múltiplas formas, criando plataformas, pa-

redes, escadas e terreno variado. Esta modularidade é fundamental para iteração rápida de *level design*.

3.3.2 *Backgrounds* com *Parallax Effect*

Para criar profundidade visual e atmosfera imersiva, foram integrados dois *packs* de *backgrounds*.

O *Free Sky with Clouds Background Pixel Art Set* [18] fornece céus diurnos com nuvens em movimento, adequados para níveis exteriores ou atmosferas luminosas. As diferentes camadas de nuvens podem ser movidas a velocidades distintas para criar um efeito de *parallax* convincente.

O *Free Crystal Cave Pixel Art Backgrounds* [19] oferece ambientações subterrâneas com cristais luminescentes e formações rochosas, ideais para níveis de caverna que contrastem visualmente com ambientes exteriores.

O *parallax scrolling* é implementado no *Godot* através dos nodos *ParallaxBackground* e *ParallaxLayer*. Cada camada move-se a uma fração da velocidade da câmara (definida pela propriedade *motion_scale*), criando a ilusão de profundidade tridimensional num espaço 2D.

3.4 Recursos Auditivos

A música ambiente foi obtida através do *Pixabay Music* [20], uma plataforma que disponibiliza música *royalty-free* sob licenças *Creative Commons*. A seleção musical focou-se em criar uma atmosfera apropriada para cada tipo de nível.

Níveis exteriores utilizam faixas com instrumentação orquestral leve e melodias ascendentes que transmitem aventura e otimismo. Níveis de caverna empregam música mais sombria com instrumentos graves e reverberação, criando tensão e mistério.

A integração de áudio no *Godot* é realizada através de nodos *AudioStreamPlayer*, *AudioStreamPlayer2D* (para sons posicionais) e *AudioStreamPlayer3D*. A gestão de múltiplas faixas permite transições suaves entre níveis e controlo de volume através do sistema de *AudioBus*.

3.5 Justificação das Escolhas Tecnológicas

As escolhas tecnológicas foram orientadas por critérios de acessibilidade, qualidade e eficiência de desenvolvimento. O *Godot* oferece capacidades profissionais sem barreiras financeiras, o que é essencial para um projeto académico. A documentação extensa e os tutoriais da comunidade reduziram significativamente a curva de aprendizagem.

A utilização de *asset packs* da comunidade *indie* permitiu alcançar qualidade visual e auditiva profissional sem necessidade de competências avançadas em arte ou composição musical. Esta abordagem é representativa do desenvolvimento *indie* contemporâneo, onde a colaboração e partilha de recursos são fundamentais.

A coerência estética entre os diferentes *asset packs* foi uma consideração importante. Todos os recursos visuais partilham o estilo *pixel art* com paletas de cores compatíveis, garantindo que o jogo apresente uma identidade visual unificada apesar de utilizar *assets* de fontes distintas.

3.6 Conclusões

O conjunto de tecnologias e ferramentas selecionadas forneceu uma base sólida para o desenvolvimento de *Knights Journey*. O *Godot Engine* provou ser uma plataforma capaz e acessível para a criação de jogos de plataforma 2D, enquanto os recursos da comunidade *indie* permitiram focar em aspetos fundamentais de *game design* e implementação. Estas escolhas tecnológicas alinharam-se perfeitamente com os objetivos do projeto, permitindo o desenvolvimento eficiente de um jogo funcional e visualmente coeso.

Capítulo

4

Implementação e Testes

4.1 Introdução

Este capítulo descreve o processo de implementação e validação do jogo *Knights Journey*, um jogo de plataformas 2D que apresenta desafios progressivamente mais complexos ao longo dos níveis. O jogo combina mecânicas clássicas do género com elementos adicionais de dificuldade, promovendo uma experiência que exige precisão, adaptação e aprendizagem contínua por parte do jogador.

Para além do aspeto mecânico, *Knights Journey* incorpora uma narrativa simples e implícita, integrada diretamente no percurso do jogador.

4.2 Movimento do Jogador e *Ice Level*

O movimento do jogador é um dos elementos centrais da experiência de jogo em *Knights Journey*. A implementação teve como principal objetivo garantir controlos responsivos e previsíveis, características fundamentais em jogos de plataforma 2D. A personagem do jogador foi implementado recorrendo ao nodo (nó) `CharacterBody2D`, beneficiando do sistema de física integrado do *Godot Engine* para gestão de colisões e movimento.

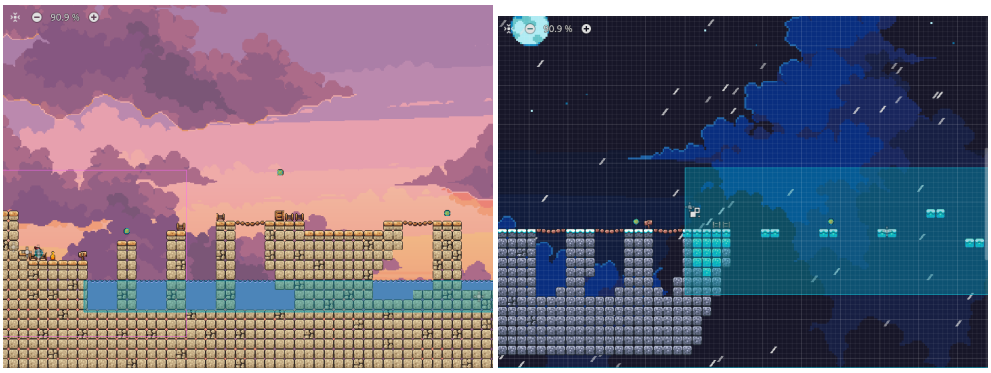

Figura 4.1: Comparação entre um nível com superfícies normais e um nível de gelo (*Ice Level*).

Na Figura 4.1 é possível observar a diferença visual entre um nível com superfícies normais e o *Ice Level*. Para além da alteração estética, esta distinção corresponde a um comportamento físico distinto do jogador, implementado ao nível da lógica de movimento, conforme descrito nos excertos de código abaixo.

4.2.1 Movimento base e resposta ao *input*

O movimento base do jogador é composto por deslocamento horizontal, salto e aplicação contínua de gravidade. O sistema responde diretamente ao *input* do utilizador, utilizando aceleração imediata no solo para garantir uma sensação de controlo precisa. O salto apenas é permitido quando o jogador se encontra em contacto com o chão, prevenindo comportamentos indesejados em pleno ar.

A animação do personagem é atualizada dinamicamente de acordo com o estado do movimento (parado, a correr ou em salto), reforçando o *feedback* visual ao jogador e aumentando a perceção de responsividade.

4.2.2 Introdução do *Ice Level*

Para diversificar a jogabilidade e introduzir novos desafios, foi implementado um nível especial designado *Ice Level*. Neste tipo de nível, as superfícies de gelo alteram o comportamento do movimento horizontal do jogador, reduzindo a fricção e aumentando a inércia. Esta mecânica obriga o jogador a antecipar movimentos e ajustar o tempo de reação, contrastando com o controlo mais direto dos níveis normais.

O estado de contacto com gelo é gerido através da variável booleana `on_ice`, definida externamente por áreas específicas do nível, permitindo separação clara entre lógica do ambiente e lógica do jogador.

```
func set_ice(value: bool) -> void:
    on_ice = value
```

Excerto de Código 4.1: Alternância do estado de movimento em superfícies de gelo.

O Excerto de Código 4.1 apresenta a função responsável por alternar o estado de contacto com superfícies de gelo. Esta função é invocada por áreas específicas do nível, permitindo que o ambiente influencie o comportamento do jogador sem introduzir dependências diretas entre o nível e a lógica da personagem.

4.2.3 Movimento em superfícies de gelo

Quando o jogador se encontra sobre gelo, o movimento horizontal passa a ser controlado por aceleração progressiva e fricção reduzida. Em vez de uma velocidade instantânea, a aceleração é aplicada gradualmente, permitindo que a personagem deslize mesmo após o jogador deixar de fornecer *input*.

```
if on_ice:
    if is_on_floor():
        if direction != 0:
            velocity.x += direction * ice_acceleration * delta
        else:
            velocity.x = move_toward(velocity.x, 0, ice_friction * delta
                )
    else:
        if direction != 0:
            velocity.x += direction * ice_air_acceleration * delta

    velocity.x = clamp(velocity.x, -max_ice_speed, max_ice_speed)
```

Excerto de Código 4.2: Movimento horizontal do jogador em superfícies de gelo.

No Excerto de Código 4.2 observa-se a implementação do movimento horizontal em superfícies de gelo. Em contraste com o movimento normal, a velocidade horizontal é modificada progressivamente através de aceleração e fricção reduzida, resultando num efeito de deslizamento. A limitação da velocidade máxima garante previsibilidade e evita aceleração excessiva.

A existência de um controlo aéreo reduzido durante o gelo mantém o movimento previsível sem eliminar completamente a capacidade de correção em pleno salto. A velocidade máxima é limitada através de `clamp`, evitando aceleração excessiva e garantindo estabilidade no comportamento da personagem.

4.2.4 Comparação com movimento normal

Em superfícies normais, o movimento horizontal é mais direto e responsivo, definindo a velocidade do jogador de forma imediata com base no *input* recebido. Esta diferença clara entre superfícies normais e gelo cria variedade na jogabilidade e contribui para uma curva de aprendizagem progressiva ao longo dos níveis.

```
else:
    if direction != 0:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)
```

Excerto de Código 4.3: Movimento horizontal do jogador em superfícies normais.

O Excerto de Código 4.3 ilustra o comportamento do movimento em superfícies normais, onde a velocidade horizontal responde de forma imediata ao *input* do jogador. A comparação direta entre este excerto e o Excerto 4.2 evidencia a diferença clara de controlo e reforça a variedade introduzida pelo *Ice Level*.

4.2.5 Impacto no *game feel* e testes

A introdução do *Ice Level* demonstrou a importância de pequenos ajustes nos parâmetros de movimento para alterar significativamente a experiência do jogador. Valores como aceleração, fricção e velocidade máxima foram ajustados iterativamente através de testes de jogabilidade, até alcançar um equilíbrio entre desafio e controlo. Esta mecânica evidencia como variações subtis na física podem ser utilizadas como ferramenta de *design* para criar níveis distintos sem introduzir novas ações ou controlos.

4.3 Sistema de Som e Áudio

O som desempenha um papel fundamental na criação de imersão e *feedback* ao jogador em *Knights Journey*. A implementação do sistema de áudio teve como principal objetivo reforçar a ambientação dos níveis e fornecer resposta imediata às ações do jogador, como saltos e recolha de *pickups*.

O sistema de áudio foi estruturado de forma modular, distinguindo música ambiente e *sound effects* (SFX), permitindo controlo independente do volume e facilitando ajustes futuros.

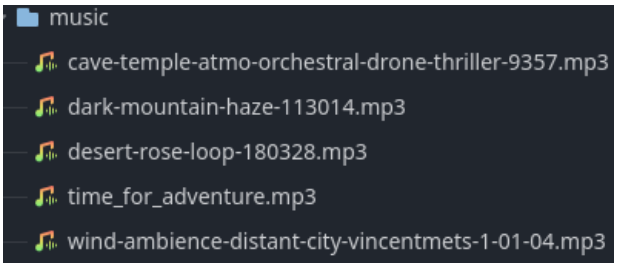

Figura 4.2: Conjunto de faixas de música ambiente utilizadas nos diferentes níveis.

Na Figura 4.2 é possível observar as diferentes faixas de música ambiente utilizadas no jogo. Cada nível possui uma música específica, selecionada de acordo com a sua temática visual e atmosfera, contribuindo para diferenciar ambientes exteriores, cavernas ou zonas de maior tensão.

4.3.1 Música Ambiente por Nível

Cada nível contém um nodo dedicado à reprodução de música ambiente, normalmente um `AudioStreamPlayer`, instanciado diretamente na cena do nível. Esta abordagem permite que a música seja automaticamente iniciada ao carregar o nível e terminada aquando da sua remoção da árvore de cenas.

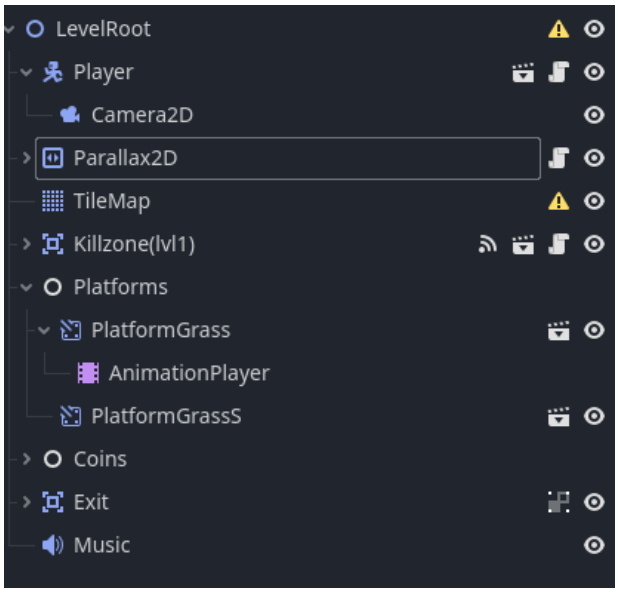

Figura 4.3: Nodo de música ambiente integrado na estrutura do nível.

A Figura 4.3 mostra a integração da música ambiente na árvore de cenas do nível. Ao associar a música diretamente ao nível, garante-se que cada cenário possui identidade sonora própria, sem necessidade de lógica adicional no gestor central do jogo.

4.3.2 Efeitos Sonoros e *Pickups*

Para além da música ambiente, foram implementados SFX para ações pontuais do jogador, como saltos e recolha de moedas. Estes efeitos sonoros são reproduzidos através de nodos `AudioStreamPlayer2D`, permitindo eventual extensão para áudio posicional.

Figura 4.4: Ficheiros de SFX utilizados para ações do jogador e *pickups*.

A Figura 4.4 apresenta alguns dos SFX utilizados no jogo, incluindo sons associados à recolha de moedas e a ações do jogador. Estes sons fornecem *feedback* imediato, tornando as interações mais satisfatórias e claras.

No caso das moedas, o efeito sonoro é reproduzido no momento da recolha, conjuntamente com a animação visual do *pickup*.

```
if pickup_sound:
    pickup_sound.play()
```

Excerto de Código 4.4: Reprodução do SFX aquando da recolha de uma moeda.

Este excerto de código demonstra a simplicidade da integração dos efeitos sonoros, mantendo a lógica do *pickup* independente do sistema global de áudio.

4.3.3 Gestão de Áudio com *Audio Buses*

A gestão do áudio é realizada através do sistema de *Audio Buses* do *Godot Engine*, permitindo separar música e efeitos sonoros em canais distintos. Esta separação facilita o controlo individual de volume e a aplicação futura de efeitos como filtros ou atenuação.

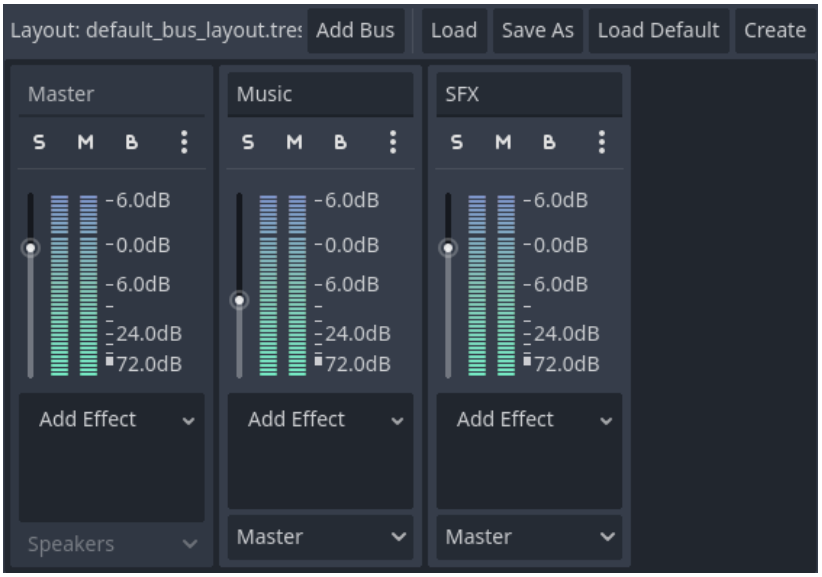

Figura 4.5: Configuração dos *Audio Buses* para música e SFX.

Na Figura 4.5 observa-se a configuração dos *buses* de áudio, nomeadamente `Music` e SFX, ambos encaminhados para o *bus* `Master`. Esta organização garante equilíbrio sonoro e permite ajustar separadamente os níveis de música ambiente e SFX, contribuindo para uma experiência auditiva mais controlada e consistente.

4.4 *Parallax Effect*

De forma a enriquecer a componente visual do jogo e aumentar a sensação de profundidade dos cenários, foi implementado um efeito de *parallax scrolling* nos níveis de *Knights Journey*. Este efeito baseia-se no movimento de múltiplas camadas de fundo a velocidades distintas relativamente à câmara, criando a ilusão de profundidade num ambiente bidimensional.

As imagens utilizadas para o *parallax* foram inicialmente ajustadas para preencher a resolução base do jogo (1920×1080). Posteriormente, foi aplicado um redimensionamento uniforme (*scaling*) de forma a preservar a qualidade visual e evitar excessiva pixelização, garantindo coerência estética com o restante estilo *pixel art* do jogo.

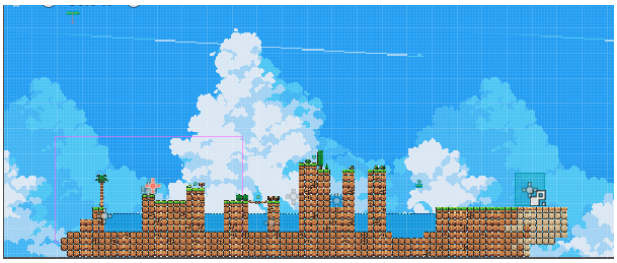

Figura 4.6: Exemplo das camadas de fundo utilizadas no efeito de *parallax*.

A Figura 4.6 ilustra as diferentes camadas de fundo que compõem o efeito de *parallax*, onde cada camada representa um plano distinto da cena. As camadas mais distantes movem-se de forma mais lenta, enquanto as camadas mais próximas apresentam um movimento mais pronunciado, reforçando a sensação de profundidade.

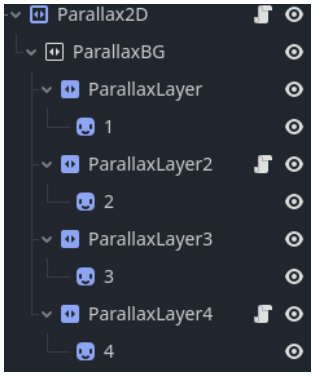

Figura 4.7: Estrutura do efeito de *parallax* na árvore de cenas do *Godot Engine*.

A Figura 4.7 apresenta a integração do efeito de *parallax* na árvore de cenas do *Godot Engine*. O efeito é implementado através de um nodo `ParallaxBackground`, que contém múltiplos nodos `ParallaxLayer`, permitindo controlar individualmente o movimento de cada camada de fundo.

4.4.1 *Scripts* de Movimento das Camadas

O movimento das camadas do *parallax* é controlado por dois *scripts* distintos, `cloudLayer1st.gd` e `cloudLayer.gd`, associados a diferentes `ParallaxLayer`. Esta separação permite definir velocidades distintas para conjuntos específicos de camadas, aumentando a sensação de profundidade e dinamismo visual.

```
extends ParallaxLayer

@onready var speed: int = -10

func _process(delta) -> void:
    self.motion_offset.x += speed * delta
```

Excerto de Código 4.5: Script `cloudLayer1st.gd` responsável pelo movimento das camadas mais distantes.

O *script* apresentado no Excerto de Código 4.5 controla o movimento horizontal das camadas mais distantes, utilizando uma velocidade reduzida. Esta escolha reflete o comportamento esperado de objetos afastados do plano principal, que aparentam mover-se mais lentamente relativamente à câmara.

```
extends ParallaxLayer

@onready var speed: int = -15

func _process(delta) -> void:
    self.motion_offset.x += speed * delta
```

Excerto de Código 4.6: Script `cloudLayer.gd` responsável pelo movimento das camadas mais próximas.

No segundo *script*, a velocidade aplicada é superior, resultando num deslocamento mais rápido das camadas de fundo mais próximas do jogador. A atualização do `motion_offset` em cada *frame* permite um movimento contínuo e suave, independente da taxa de atualização do jogo.

A combinação destas velocidades distintas cria um efeito visual consistente e natural, contribuindo para uma apresentação mais rica dos cenários sem impactar significativamente o desempenho do jogo.

4.5 *Pickups* e Sistema de Recolha

Os *pickups* representam elementos colecionáveis que incentivam a exploração dos níveis e reforçam a progressão do jogador. Em *Knights Journey*, as moedas constituem o principal tipo de *pickup*, sendo implementadas como nodos `Area2D`, permitindo deteção simples e eficiente da colisão com o jogador.

Figura 4.8: Exemplo de moedas posicionadas ao longo de um nível.

A Figura 4.8 exemplifica a colocação de moedas ao longo de um nível, demonstrando como estes elementos são utilizados para guiar o jogador e incentivar a exploração de caminhos alternativos.

Cada moeda é responsável apenas por detetar a colisão e emitir um sinal personalizado (`collected`), seguindo o princípio de separação de responsabilidades. A lógica associada à pontuação é gerida centralmente pelo nó `Main`, conforme descrito na Secção 4.8.

```
extends Area2D

signal collected

@onready var pickup_animation: AnimationPlayer = $PickupAnimation
@onready var pickup_sound: AudioStreamPlayer2D = $PickupSound

func _on_body_entered(body: Node2D) -> void:
    emit_signal("collected")

    if pickup_sound:
        pickup_sound.play()

    if pickup_animation:
        pickup_animation.play("pickup")
```

Excerto de Código 4.7: Implementação da moeda e emissão do sinal de recolha.

No Excerto do código 4.7 apresenta a implementação de uma moeda (*pickup*). Quando o jogador entra na área da moeda, o sinal `collected` é emitido e intercetado pelo `Main`, que incrementa a pontuação do jogador. Esta abordagem permite que as moedas permaneçam completamente independentes do sistema de pontuação, facilitando reutilização e extensão futura para outros tipos de *pickups*.

Para reforçar o *feedback* ao jogador, a recolha de uma moeda é acompanhada por uma animação visual e um efeito sonoro, proporcionando confirmação imediata da ação. A animação de recolha permite também sincronizar a remoção visual da moeda do nível, aumentando a sensação de fluidez e *polish* do jogo.

Este sistema baseado em sinais simplifica a integração de múltiplas moedas num nível, uma vez que todas seguem a mesma lógica e são conectadas dinamicamente durante a fase de configuração do nível.

4.6 *KillZones* e *Restart* do Nível

As *KillZones* são áreas invisíveis utilizadas para detetar situações de morte do jogador, como quedas fora dos limites do mapa ou contacto com zonas letais. Em *Knights Journey*, estas zonas são implementadas com um nodo `Area2D`, permitindo detetar a entrada do jogador através do sinal `body_entered` e desencadear o *restart* do nível de forma controlada.

Figura 4.9: Exemplo de posicionamento de uma *KillZone* na zona inferior do nível.

Na Figura 4.9 é ilustrado o posicionamento de uma *KillZone* na zona inferior do nível, destinada a detetar quedas fora dos limites jogáveis e outras situações de morte do jogador.

O *restart* é realizado preferencialmente através do nó `Main`, identificado por pertença ao grupo `Main`. Desta forma, o jogo reutiliza o mecanismo central de gestão e carregamento de níveis descrito na Secção 4.8, garantindo consistência na reposição do estado. Como mecanismo de contingência, caso o `Main` não esteja disponível, é efetuado um *reload* da cena atual, assegurando que o jogo não fica num estado inválido.

```
extends Area2D

@onready var timer: Timer = $Timer

func _on_body_entered(body: Node2D) -> void:
    Engine.time_scale = 0.5
    timer.start()

func _on_timer_timeout() -> void:
    Engine.time_scale = 1.0

    var main := get_tree().get_first_node_in_group("Main")
    if main != null and main.has_method("restart_current_level"):
        await main.restart_current_level()
        return

    get_tree().reload_current_scene()
```

Excerto de Código 4.8: Implementação da *KillZone* com efeito de *slow-motion* e *restart* do nível.

O Excerto de Código 4.8 demonstra a lógica associada à `Killzone` que aplica um pequeno efeito de *slow-motion* no momento da morte, reduzindo temporariamente a escala do tempo do jogo (`Engine.time_scale`), o que reforça o *feedback* ao jogador e torna o evento mais percetível. De seguida, um `Timer` introduz um atraso curto antes de reiniciar o nível, permitindo que o jogador reconheça o erro antes de regressar ao ponto inicial.

4.7 Armadilhas Dinâmicas: *Stalactites*

As *stalactites* constituem armadilhas dinâmicas utilizadas para aumentar a tensão e o desafio em determinados níveis de *Knights Journey*. Estas armadilhas reagem à presença do jogador e introduzem elementos de antecipação, exigindo atenção ao ambiente e capacidade de reação rápida.

As *stalactites* são implementadas como nodos `RigidBody2D`, permitindo que o motor de física do *Godot* trate naturalmente a queda, colisões e interações com o jogador e o cenário. O seu comportamento é controlado através de estados bem definidos: inativo, aviso (*warning*) e queda.

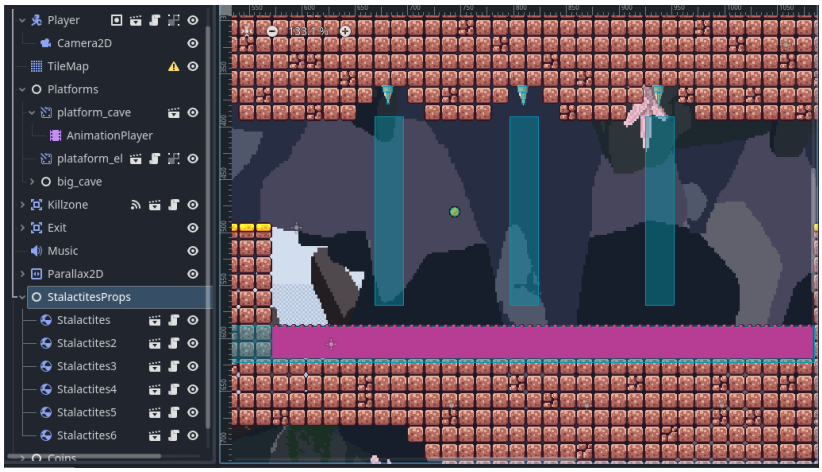

Figura 4.10: Posicionamento de uma stalactite no teto do nível com área de deteção do jogador.

A Figura 4.10 apresenta o posicionamento de três *stalactites* no teto do nível, bem como a respetiva área de deteção do jogador, responsável por ativar a armadilha.

4.7.1 Estado inicial e preparação

No estado inicial, a *stalactite* encontra-se suspensa e completamente inativa.

```
func _ready() -> void:
    freeze = true
    sleeping = true
    base_pos = global_position
    trigger.body_entered.connect(_on_trigger_body_entered)
```

Excerto de Código 4.9: Configuração inicial da *stalactite*.

No Excerto de Código 4.9 é apresentada a configuração inicial da *stalactite*, onde o corpo físico é mantido inativo através das propriedades `freeze` e `sleeping`. Esta abordagem garante que a armadilha permanece estática até ser explicitamente ativada.

4.7.2 Deteção do jogador e aviso prévio

A ativação da armadilha ocorre quando o jogador entra na área de deteção (`Area2D`) associada à *stalactite*. Para evitar ativações repetidas, é utilizado um mecanismo de bloqueio através da variável `triggered` como vemos no Excerto de Código 4.10 abaixo. Após a deteção, inicia-se uma fase de aviso curto, durante a qual a *stalactite* apresenta sinais visuais de instabilidade.

```
func _on_trigger_body_entered(body: Node) -> void:
    if triggered or not body.is_in_group("Player"):
        return

    triggered = true
    trigger.set_deferred("monitoring", false)
    _start_warning()
```

Excerto de Código 4.10: Ativação da *stalactite* ao detetar o jogador.

Durante a fase de aviso, a stalactite executa um ligeiro movimento de oscilação horizontal e pode sofrer uma alteração temporária de cor, funcionando como *telegraphing* do perigo iminente.

```
func _process(_delta: float) -> void:
    if warning:
        var t := Time.get_ticks_msec() / 1000.0
        global_position = base_pos + Vector2(sin(t * shake_speed) *
            shake_pixels, 0)
```

Excerto de Código 4.11: Efeito visual de aviso antes da queda.

4.7.3 Queda e libertação da física

Após um curto intervalo de tempo, a *stalactite* é libertada, passando a ser afetada pela gravidade. A queda é suavizada através de um fator de escala de gravidade inferior ao valor padrão, criando uma descida mais legível e menos abrupta para o jogador.

```
func _drop_deferred() -> void:
    warning = false
    global_position = base_pos
    gravity_scale = fall_gravity_scale
    freeze = false
    sleeping = false
    apply_impulse(Vector2(0, drop_impulse))
```

Excerto de Código 4.12: Libertação da *stalactite* e início da queda.

O Excerto de Código 4.12 apresenta o momento de libertação da *stalactite*; a utilização de um impulso inicial garante que ela inicie o movimento de forma consistente, evitando comportamentos indeterminados do sistema de física.

4.7.4 Impacto no design de níveis

As *stalactites* foram posicionadas estrategicamente em zonas de passagem frequente, obrigando o jogador a manter atenção ao ambiente superior e a temporizar o movimento. O breve aviso antes da queda equilibra o nível de dificuldade, tornando a armadilha justa e previsível sem eliminar o desafio.

Esta abordagem demonstra como armadilhas dinâmicas podem ser utilizadas para enriquecer o *gameplay* recorrendo a princípios simples de física, *feedback* visual e controlo temporal.

4.8 Gestão e Transição de Níveis

A gestão de níveis em *Knights Journey* é centralizada no nó principal do jogo, designado `Main`, responsável por controlar o estado global da aplicação, incluindo o nível atual, a pontuação do jogador e as transições entre cenas. Cada nível do jogo é implementado como uma cena independente do *Godot Engine*, localizada na diretoria `scenes/levels`, incluindo uma cena de introdução (`intro.tscn`) e uma cena final de créditos (`credits.tscn`).

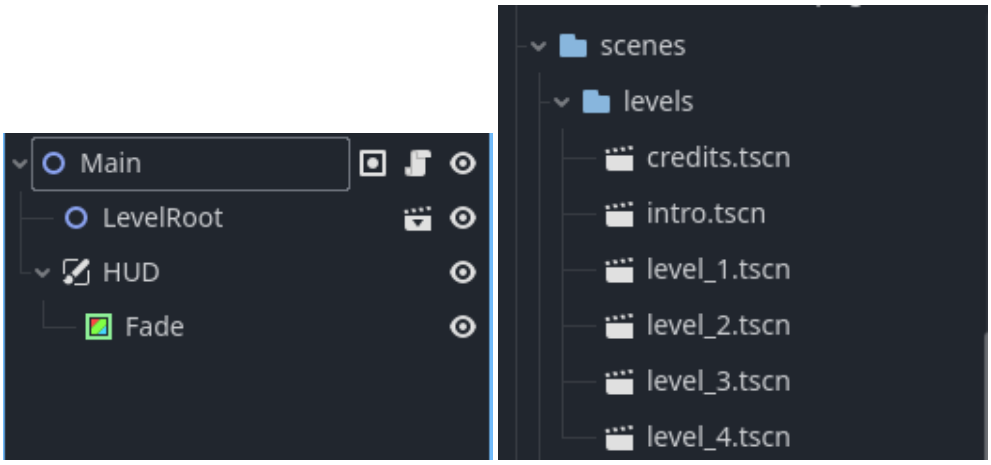

Figura 4.11: Organização dos níveis como cenas independentes.

A Figura 4.11 ilustra a organização dos níveis como cenas independentes, incluindo a cena de introdução, os níveis jogáveis e a cena final de créditos. Isto permite fácil manutenção do projeto e a adição de novos níveis sem necessidade de alterações significativas

4.8.1 Ciclo de vida dos níveis e nó `LevelRoot`.

Para garantir que apenas um nível se encontra ativo em cada momento, antes de instanciar uma nova cena, o nível anterior é removido da árvore de cenas.

```
func _load_level(level_number: int) -> void:
    await _fade(1.0)
    if current_level_root:
        current_level_root.queue_free()
        current_level_root = null

    var path := "res://scenes/levels/level_%s.tscn" % level_number
    var packed := load(path)
    if packed == null:
        level = 1
        packed = load("res://scenes/levels/level_1.tscn")
        if packed == null:
            return

    current_level_root = packed.instantiate()
    current_level_root.name = "LevelRoot"
    add_child(current_level_root)
    _setup_level(current_level_root)
    await _fade(0.0)
```

Excerto de Código 4.13: Carregamento dinâmico e substituição do nível atual.

O Excerto de Código 4.13 demonstra o processo de substituição do nível atual. `Main` mantém uma referência ao nível através da variável `current_level_root`. Sempre que ocorre uma transição, o nível anterior é removido da árvore de cenas utilizando `queue_free()`, evitando a duplicação de objetos e possíveis problemas de memória.

4.8.2 Transições com efeito de *fade*.

De forma a melhorar a experiência do utilizador, todas as transições entre níveis são acompanhadas por um efeito de *fade*. Este efeito é implementado recorrendo a um nodo `ColorRect` sobreposto ao ecrã e a um `Tween`, permitindo uma transição visual suave entre estados.

```
func _fade(to_alpha: float) -> void:
    var tween := create_tween()
    tween.tween_property(fade, "modulate:a", to_alpha, 1.5)
    await tween.finished
```

Excerto de Código 4.14: Implementação do efeito de *fade* entre níveis.

Como vemos no Excerto de Código 4.14, a utilização de `await` garante que a execução da lógica de jogo é sincronizada com o término da animação, evitando mudanças abruptas de cena.

4.8.3 Cena de introdução e início do jogo.

O jogo inicia-se com o carregamento da cena de introdução. Esta cena comunica com o `Main` através da emissão de um sinal (`start_game`)

```
func _show_intro() -> void:
    await _fade(1.0)
    current_level_root = intro_scene.instantiate()
    current_level_root.name = "LevelRoot"
    add_child(current_level_root)

    if current_level_root.has_signal("start_game"):
        current_level_root.connect("start_game", _start_game_from_intro)

    await _fade(0.0)
```

Excerto de Código 4.15: Ligação do sinal de início da cena de introdução.

Quando o sinal é emitido, o jogo inicia no primeiro nível, reutilizando o mesmo mecanismo de carregamento aplicado nas restantes transições. Este mecanismo permite iniciar o jogo de forma independente, sem que a lógica da introdução dependa diretamente do sistema de níveis.

4.8.4 Configuração do nível e ligação de eventos.

Após o carregamento de um nível, é executada uma fase de configuração responsável pela ligação dos sinais emitidos por elementos do nível, como a saída e os objetos colecionáveis.

```
func _setup_level(level_root: Node) -> void:
    var exit = level_root.get_node_or_null("Exit")
    if exit:
        exit.body_entered.connect(_on_exit_body_entered)

    var coins = level_root.get_node_or_null("Coins")
    if coins:
        for coin in coins.get_children():
            coin.collected.connect(increase_score)
```

Excerto de Código 4.16: Configuração do nível e ligação de sinais.

No Excerto de Código 4.16 observa-se a fase de configuração do nível. Ao abordar destra forma permite que cada nível seja autónomo e facilmente reutilizável.

4.8.5 Progressão entre níveis e créditos finais.

A progressão do jogo é desencadeada quando o jogador colide com o nó `Exit`. Caso o nível atual corresponda ao último nível definido, o jogo transita para a cena de créditos; caso contrário, o índice do nível é incrementado e a cena seguinte é carregada.

Esta lógica centraliza as regras de progressão do jogo num único ponto, simplificando futuras alterações à estrutura ou número de níveis.

4.9 Testes e Validação

A fase de testes e validação desempenhou um papel fundamental no desenvolvimento de *Knights Journey*, permitindo avaliar progressivamente a jogabilidade, estabilidade e coerência das mecânicas implementadas. Os testes foram realizados de forma iterativa ao longo de diferentes versões do jogo, possibilitando a identificação de problemas e a introdução de melhorias baseadas em *feedback* contínuo.

4.9.1 Versão Inicial do Protótipo

A versão inicial do jogo consistia num único nível onde foram implementadas as mecânicas base de um jogo de plataformas 2D. Esta versão incluía plataformas estáticas e móveis, zonas de morte simples e moedas colecionáveis. As *killzones* eram implementadas de forma rudimentar, recorrendo a uma `WorldBoundaryShape2D`, onde a queda fora do mundo resultava na morte imediata do jogador.

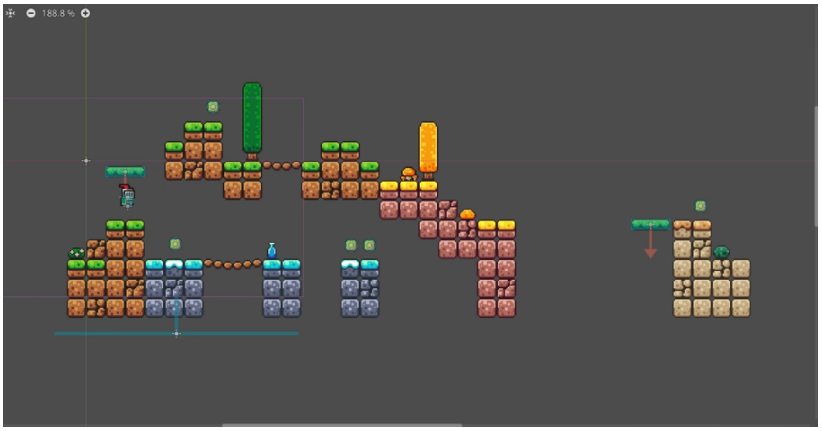

Figura 4.12: Protótipo inicial do jogo com um único nível e mecânicas base de plataformas 2D.

Na Figura 4.12 observa-se a simplicidade estrutural da versão inicial, que serviu como base para a evolução posterior do projeto. Nesta fase, as moedas não eram persistentes nem reiniciadas após a morte do jogador, funcionando apenas como prova de conceito. Apesar das limitações, esta versão permitiu validar o controlo da personagem, a interação com o ambiente e a viabilidade técnica das mecânicas planeadas. Os testes desta versão foram realizados exclusivamente pelo autor do projeto, servindo como base para a definição da estrutura geral do jogo.

4.9.2 Segunda Versão: Estruturação do Jogo

A segunda versão representou uma evolução significativa do projeto, marcada pela introdução da estrutura base do jogo. Nesta fase, o jogo passou a ser organizado em múltiplos níveis distintos, cada um introduzindo novas mecânicas e desafios progressivos.

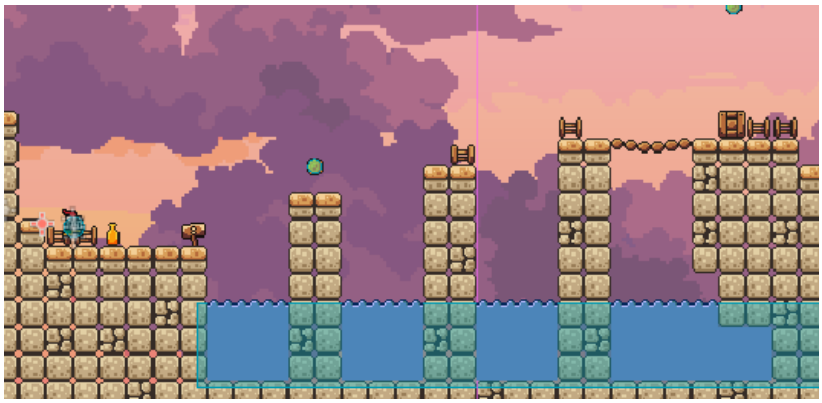

Figura 4.13: Evolução do jogo com introdução de *killzones* mais elaboradas e estruturação dos níveis.

Foram implementadas *killzones* mais elaboradas, como zonas de água, consolidando o conceito descrito na Secção 4.6. Adicionalmente, foi criado o sistema central de gestão do jogo (*Main Root*), responsável pela transição entre níveis e gestão do estado global, conforme apresentado na Secção 4.8.

As moedas passaram a ser contabilizadas corretamente, embora ainda não fossem reiniciadas após a morte do jogador, conforme descrito na Secção 4.5. Nesta versão foi também introduzido o efeito de *parallax*, descrito na Secção 4.4, contribuindo para uma melhoria significativa da apresentação visual.

Esta versão foi testada por terceiros, nomeadamente amigos do autor, que forneceram *feedback* essencial para melhorar o *game feel*, ajustar valores de movimento e identificar situações de frustração ou dificuldade excessiva.

4.9.3 Versão Final: Consolidação e Refinamento

A versão final do jogo concentrou-se na consolidação das mecânicas existentes e na introdução de novos desafios. Nesta fase foi implementado o reinício correto das moedas após a morte do jogador, completando o sistema de *pickups* descrito na Secção 4.5.

Foram adicionados níveis especiais, como o *Ice Level*, que introduz uma variação significativa no comportamento do movimento do jogador, conforme detalhado na Secção 4.2. Adicionalmente, foram integradas armadilhas dinâmicas, nomeadamente as *stalactites*, descritas na Secção 4.7, aumentando a complexidade e exigência dos desafios.

A criação destes desafios mais avançados resultou diretamente do *feedback* obtido nos testes realizados na versão anterior, refletindo uma abordagem iterativa ao *design* do jogo. Foram ainda implementadas a cena de introdução e a cena de créditos, conforme descrito na Secção 4.8, contribuindo para uma experiência mais completa e coesa.

Figura 4.14: Cena de introdução e cena de créditos da versão final do jogo.

Esta versão final foi testada maioritariamente pelo autor do projeto e pelo orientador, focando-se na validação final da jogabilidade, estabilidade e coerência global do jogo.

4.9.4 Considerações Finais sobre os Testes

O processo de testes iterativos revelou-se essencial para a evolução de *Knights Journey*. A progressão desde um protótipo simples até uma versão final mais complexa permitiu validar decisões técnicas e de *design*, demonstrando a importância do *feedback* externo e da iteração contínua no desenvolvimento de jogos de plataformas 2D.

4.10 Conclusões

A implementação e os testes permitiram validar as principais mecânicas de *Knights Journey*, resultando num protótipo funcional e jogável. As soluções adotadas demonstram a eficácia do *Godot Engine* para desenvolvimento de jogos de plataforma 2D e fornecem uma base sólida para futuras extensões do projeto.

Capítulo

5

Conclusões e Trabalho Futuro

5.1 Conclusões Principais

O desenvolvimento de *Knights Journey* permitiu explorar de forma abrangente o processo de criação de um jogo de plataforma 2D, desde a conceção inicial até à implementação e validação de mecânicas funcionais. O projeto demonstrou que é possível desenvolver uma experiência de jogo coesa e jogável recorrendo a ferramentas *open-source* e a recursos da comunidade *indie*, confirmando a viabilidade do desenvolvimento de videojogos em contexto académico.

Uma das conclusões centrais do projeto prende-se com a importância do *game feel* na experiência do jogador. Tal como evidenciado na Secção 4.2, a implementação de movimento responsivo e física bem calibrada revelou-se mais exigente do que inicialmente previsto. Pequenos ajustes em parâmetros como aceleração, fricção, força do salto e velocidade máxima produziram alterações significativas na sensação de controlo, particularmente visíveis na comparação entre superfícies normais e o *Ice Level*. Esta observação confirma que, em jogos de plataforma, a qualidade da experiência reside frequentemente em detalhes subtis da implementação.

O *Godot Engine* revelou-se uma escolha particularmente adequada para o desenvolvimento do projeto. A arquitetura baseada em cenas facilitou uma organização modular e reutilizável, visível na gestão centralizada dos níveis descrita na Secção 4.8. O sistema de física 2D integrado simplificou a implementação de colisões, movimento e armadilhas dinâmicas, como as *stalactites* apresentadas na Secção 4.7. Adicionalmente, a linguagem *GDScript* permitiu uma implementação clara e concisa da lógica de jogo, como demonstrado nos excertos de código relativos ao movimento em gelo (Excertos 4.2

e 4.3) e às *KillZones* (Excerto 4.8).

O *level design* emergiu como um processo simultaneamente criativo e técnico. A criação de níveis equilibrados exigiu compreensão aprofundada das mecânicas disponíveis e das limitações do jogador, bem como iteração contínua baseada em testes. A progressão gradual de dificuldade, aliada à introdução de novas mecânicas como o gelo e as armadilhas dinâmicas, contribuiu para uma curva de aprendizagem progressiva, conforme discutido na Secção 4.9.

A integração de *assets* externos revelou-se uma estratégia eficaz para alcançar qualidade visual e auditiva consistente. A utilização de pacotes de recursos visuais e musicais permitiu concentrar o esforço no design e na programação, mantendo uma apresentação cuidada, como evidenciado pelo sistema de *parallax scrolling* descrito na Secção 4.4 e pelo sistema de áudio apresentado na Secção 4.3. Contudo, esta abordagem exigiu pesquisa cuidadosa para garantir coerência estética entre recursos provenientes de diferentes fontes.

O processo de testes iterativos demonstrou-se fundamental para a evolução do projeto. A progressão desde um protótipo simples até à versão final permitiu identificar limitações, ajustar mecânicas e melhorar o *game feel*, validando a importância do *feedback* externo e da iteração contínua no desenvolvimento de jogos de plataforma 2D.

De forma global, o projeto reforçou a importância do planeamento e da gestão de *scope*. Funcionalidades aparentemente simples revelaram complexidade inesperada durante a implementação, tornando essencial a definição de prioridades claras e o foco nas mecânicas *core*. A convergência bem-sucedida entre programação, design de níveis, feedback visual e auditivo e balanceamento de dificuldade foi determinante para a obtenção de um protótipo funcional e coerente.

5.2 Trabalho Futuro

Apesar de *Knights Journey* cumprir os objetivos definidos, o projeto pode ser encarado como um protótipo funcional, existindo diversas possibilidades de expansão e refinamento.

5.2.1 Sistema de Menus e Interface

A ausência de um sistema de menus completo constitui a limitação mais evidente da versão atual. A implementação de um menu principal permitiria iniciar o jogo, selecionar níveis, configurar opções de áudio e sair da aplicação. Um menu de pausa *in-game* acrescentaria controlo adicional ao jogador, exigindo gestão adequada do estado do jogo através do *pause mode* do *Godot*.

Adicionalmente, a introdução de elementos de *HUD* (*Heads-Up Display*), como contadores ou indicadores visuais, contribuiria para maior *polish* e clareza da informação apresentada ao jogador.

5.2.2 Expansão de Inimigos e Desafios

O jogo beneficiaria da introdução de maior diversidade de inimigos com padrões de comportamento distintos. Inimigos voadores, patrulhas dinâmicas e inimigos com reações condicionais à proximidade do jogador poderiam aumentar a complexidade estratégica.

A introdução de mini-*bosses* ou *boss fights* no final de níveis representaria momentos de culminação da progressão, exigindo domínio das mecânicas aprendidas e aplicação combinada de movimento, precisão e leitura de padrões.

5.2.3 Novas Mecânicas de Movimento

A adição de mecânicas avançadas de movimento expandiria significativamente as possibilidades de *level design*. Mecânicas como *wall grabbing*, *wall jumping* ou *dash* permitiriam criar secções mais verticais, desafios de precisão e abordagens alternativas à navegação dos níveis.

Estas mecânicas poderiam ser integradas de forma progressiva, mantendo a filosofia de introdução gradual de desafios adotada ao longo do projeto.

5.2.4 Aplicações e Extensões do Projeto

Para além do contexto de jogo, os sistemas desenvolvidos em *Knights Journey* podem ser reutilizados em projetos futuros. O sistema de movimento e colisão pode servir de base para outros géneros 2D, como *metroidvanias* ou *puzzle-platformers*. A arquitetura modular facilita adaptações e experimentações com novas mecânicas ou estilos de jogo.

O projeto pode igualmente funcionar como recurso educacional, demonstrando de forma prática a implementação de mecânicas de plataforma 2D no *Godot Engine*, contribuindo para a comunidade *open-source*.

5.3 Considerações Finais

Knights Journey representa mais do que a conclusão de um projeto académico: constitui uma experiência formativa completa no domínio do desenvolvimento de videojogos. As competências técnicas adquiridas, aliadas à compreensão prática de princípios de *game design*, iteração e teste, fornecem uma base sólida para projetos futuros, seja em contexto académico, *indie* ou profissional.

Bibliografia

[1] Ernest Adams. *Fundamentals of Game Design*. New Riders, 2014.

[2] Jesse Schell. *The Art of Game Design: A Book of Lenses*. CRC Press, 2019.

[3] Jason Gregory. *Game Engine Architecture*. CRC Press, 2018.

[4] Kristian Kiili. Digital game-based learning: Towards an experiential gaming model. *The Internet and Higher Education*, 8(1):13–24, 2005.

[5] Godot Engine. Godot Engine Documentation, 2024. [Online] `https://docs.godotengine.org`. Último acesso a 28 de Janeiro de 2026.

[6] Steven L. Kent. *The Ultimate History of Video Games*. Three Rivers Press, 2001.

[7] Mark J. P. Wolf. *The Video Game Explosion*. Greenwood Press, 2008.

[8] Jesper Juul. *Half-Real: Video Games between Real Rules and Fictional Worlds*. MIT Press, 2005.

[9] Brendan Keogh. Indie games and the aesthetics of difficulty. *Game Studies*, 2019.

[10] Tracy Fullerton. *Game Design Workshop*. CRC Press, 2018.

[11] Mihaly Csikszentmihalyi. *Flow: The Psychology of Optimal Experience*. Harper & Row, 1990.

[12] Jeannie Novak. *Game Development Essentials*. Delmar, 2012.

[13] Michael Nitsche. *Video Game Spaces*. MIT Press, 2008.

[14] Unity Technologies. Unity Game Engine Documentation, 2024. [Online] `https://docs.unity.com`. Último acesso a 28 de Janeiro de 2026.

[15] Epic Games. Unreal Engine Documentation, 2024. [Online] `https://docs.unrealengine.com`. Último acesso a 28 de Janeiro de 2026.

[16] itch.io. itch.io – Indie Game Hosting Platform, 2024. [Online] https://itch.io. Último acesso a 28 de Janeiro de 2026.

[17] Brackeys. Brackeys Platformer Bundle, 2020. [Online] https://brackeysgames.itch.io/brackeys-platformer-bundle. Último acesso a 28 de Janeiro de 2026.

[18] Free Game Assets. Free Sky with Clouds Background Pixel Art Set, 2021. [Online] https://free-game-assets.itch.io/free-sky-with-clouds-background-pixel-art-set. Último acesso a 28 de Janeiro de 2026.

[19] Free Game Assets. Free Crystal Cave Pixel Art Backgrounds, 2021. [Online] https://free-game-assets.itch.io/free-crystal-cave-pixel-art-backgrounds. Último acesso a 28 de Janeiro de 2026.

[20] Pixabay. Pixabay Music – Royalty Free Music, 2024. [Online] https://pixabay.com/music. Último acesso a 28 de Janeiro de 2026.